

Bachelor's Thesis Proposal

Applying Model-driven Security and Privacy for Developing Data-centric Web Applications

Assessor: Dr. Hoang Nguyen
Co-assessor: To be decided

1 Motivation

Ensuring that software systems, especially web applications, comply with security and privacy requirements is a complex and critical challenge. On the one hand, it is complex because laws, such as the GDPR [1], impose strict requirements on how organizations handle (personal) data but offer limited guidance on concrete technical solutions. On the other hand, it is critical because failing to comply with these requirements can result in substantial financial penalties.

Model-driven privacy [3] (MDP) is a software development methodology in which privacy requirements are specified as models, and enforcement mechanisms are generated automatically via model transformations. MDP enables developers to specify three interconnected declarative models: a data model describing the state space of the system, a security model capturing fine-grained access control requirements, and a privacy model expressing data protection requirements, which is a particularly challenging class of privacy requirements. From these models, the methodology automatically generates a web application together with enforcement mechanisms that make it security- and privacy-aware by design.

The authors of the aforementioned paper have built and published the tool, called *ν*ActionGUI [4], which implements the model-driven security and privacy approach. It generates a web application using the Flask [2] (Python) framework. Recently, they have improved the tool to support more features that help users better control their personal information, such as exercising their rights to erasure, rectification, and objection.

2 Objective

In this thesis, we plan to apply this tool in developing a case study web application. The web application is designed to have simple functional requirements but complex security and privacy requirements. Through developing the web application, we attempt to answer the following research questions:

1. **RQ1.** To what extent does the model-driven privacy approach effectively help developers specify privacy policies that comply with regulations?
2. **RQ2.** To what extent does the model-driven privacy approach effectively help developers enforce privacy policies that comply with regulations?
3. **RQ3.** What are the most challenging aspects of specifying and enforcing privacy policies using this approach?

3 Tasks

1. Study the existing MDP approach.
2. Study the existing tool ν ActionGUI.
3. Study the project requirements for the case study.
4. Implement the project using ν ActionGUI and provide answers to the research questions.
5. Document the design, implementation, and evaluation in the final report.

4 Deliverables

The following deliverables are due at the end of the project:

Final report Please have a look at the submission requirements in the following section. The final report should consist of an introduction, a theoretical background section, sections describing the work packages and their outcomes, and a conclusion. The report must be written in English.

Source code Complete, documented source code with a reproducible evaluation is required.

Presentation At the end of the thesis, a 20-minute presentation must be given in a thesis defense seminar. It should provide an overview and discuss the most important highlights of the work.

References

- [1] European Parliament and Council. Regulation (EU) 2016/679, 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [2] Flask. Flask Web Framework, 2023.
- [3] Srdan Krstic, Hoang Nguyen, and David Basin. Model-driven privacy. *Proc. Priv. Enhancing Technol.*, 2024(1):314–329, 2024.
- [4] Hoang Nguyen, Srdan Krstic, and David Basin. ν ActionGUI: Model-driven security and privacy, 2024.

5 Project Description

The web application to be developed is a simplified version of typical sports management platforms called SportLink. Anyone can visit the platform and freely register to access basic functionality. Registered users may also buy a premium subscription, which unlocks additional features. Training sessions are managed by trainers, while the rest of the platform's content is managed by administrators.

The main purpose of the platform is to maintain a collection of sports sessions, organized and curated by users. Registered users can create sessions, either free play or training, which they then own. A session, modeled by the **Session** entity, contains a `title`, a `description`, and scheduling details.

Other users can register to attend these sessions. Each registration is stored as an object of the **Registration** entity.

Users are modeled by the **Person** entity, which includes attributes such as `name`, `surname`, `email` (address), and `role`. Each session is associated with a sport to which users may subscribe. Sports are modeled by the **Sport** entity, which includes a `name`, a `description`, an `icon`, and is managed by the platform's administrators.

The platform also serves advertisements and recommends sports sessions to users. The **Ad** entity includes only a `content` attribute. In addition, the platform sends mass marketing messages to users and provides administrators with statistical insights into user participation and sport popularity.

The figure below shows a (simplified) class diagram representing the SportLink Platform's data model. The omitted `0..*` association end collection types can be seen in the data models provided in the templates. Note: in the *νActionGUI* data model, the `role` attribute is implicitly added to the entity that represents users in the security model.

The list of registered participants for a session must never exceed the session's defined capacity and must always include the session's owner. Additionally, if the session is a training session, the assigned trainers must hold the `TRAINER` role and be registered for the session. **You must strictly maintain these invariants in your implementation.**

When entities are created, all attributes and `0..1` association ends are initialized to null (represented with `None` in Python), while all `0..*` association ends are initialized to empty collections. This initialization behavior can make it difficult to define authorization constraints in *νActionGUI*, as one cannot reliably determine which attributes have already been set to appropriate values and can, therefore, be used to express the constraint. To address this, upon the creation of each entity, you may assume that attributes and association ends will be set in the following order (in our application, we do not have the functionality to create **Person** and **Sport**; for that reason, the order of creation for objects of these entities can be ignored):

- **Session**: `owner`, `training`, `started_at`, `title`, `description`, `sport`, `capacity`, `duration`, `registrations`
- **Registration**: `session`, `owner`, `registered_at`

If an attribute or association end is not listed above, it may not be initialized. If it is initialized, it can happen in any order with respect to other unlisted attributes or association ends. For example, a session's `owner` will always be set and, in particular, before its `sport` association end. Hence, when writing an authorization constraint on `sport`, you can safely rely on the `owner` attribute having been initialized, but not vice versa.

In this project, you will apply the concepts of model-driven security and privacy (using *νActionGUI*) to implement the required security and privacy requirements described in the following sections.

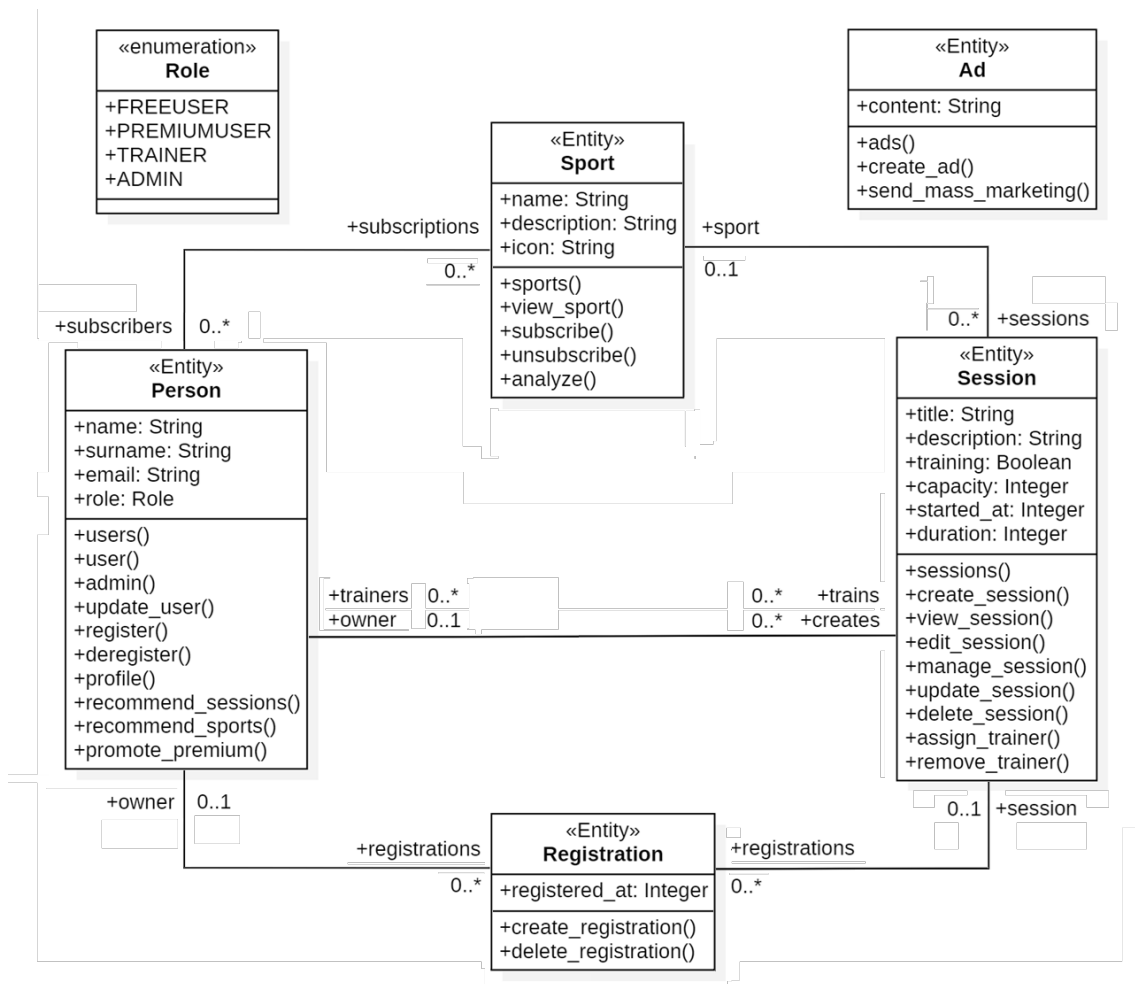


Figure 1: Simplified data model of the SportLink platform.

6 Security Requirements

In this project, we focus on authorization policies, a specific class of security requirements. These policies define a set of permissions, which are described in the following subsections. Permissions are grouped first by specific entities and then by specific user roles.

Authenticated users of the SportLink platform can hold one of the following roles: `FREEUSER`, `PREMIUMUSER`, `TRAINER`, or `ADMIN`. Anyone accessing the platform without authentication is considered to have the `VISITOR` role.

Throughout the rest of this section, whenever we refer to *users*, we mean the authenticated users of the platform. Whenever we refer to *everyone*, we mean both the authenticated and non-authenticated users (i.e., both platform users and visitors).

6.1 General Remarks

Shared permissions

Sessions in SportLink can be of two types: free play or training. To distinguish between them, everyone is allowed to read the `training` flag of a session. Each session is managed by its owner and, if applicable, its assigned trainers. In the context of a session, the owner has all the permissions that the trainers have, and the trainers have all the permissions that the registered participants have.

Association ends

In *νActionGUI*, for permissions related to adding, removing, and updating association ends, pay special attention to both ends of the association and implement all constraints *symmetrically*. For example, if there is a constraint for `update(owner)`, you must also define analogous constraints for `add(sessions)` and `remove(sessions)`.

6.2 Stateful Permissions

Session

Premium users can create freeplay sessions, while only users with the `TRAINER` role are allowed to create training sessions. Everyone can view a session's basic information, including its `title`, `description`, `capacity`, start time (`started_at`), `duration`, and the `sport` it belongs to. A session's trainer can edit its core information, with the exception of the `started_at` attribute, `owner`, and the `sport` association end. These fields cannot be modified once they have been initially set (Hint: you can determine whether an attribute or a 0..1 association-end value has been set by checking if it is null).

Any user can view the list of registered participants for a session, meaning they can read the session's registration list and the owner of each registration. They can also see who created the session and, in case of training sessions, who the assigned trainers are. Trainers may assign one of the attendants as the new trainer for their assigned session (i.e., have them become one of the session's trainers in addition to just being an attendant). However, trainers may only remove themselves from being a trainer of a session; they are not allowed to remove other trainers.

Registration

Any user may register to join a session, as long as the session has not yet started (i.e., the value of the registration's `registered_at` must be smaller than the session's `started_at`) and there are available slots (i.e., the number of current registrations is less than the session's defined `capacity`). Any user may deregister themselves from a session, unless they are the `owner` or one of the `trainers` of the session.

Person

Users can view any user's core information, including their `name`, `surname`, and `role`. They may edit their own core information, except for their `role`. Users can also view and edit their own `email` address. Additionally, each user can see the sessions they create, train, register to attend, as well as the sports they are subscribed to.

Sport

All sports, including their names, descriptions, and associated sessions, are visible to everyone. However, only administrators can view the subscriber list of a sport. Users may subscribe to or unsubscribe themselves from any sport.

Ad

Everyone can see the content of any advertisement. Only administrators are allowed to create or delete advertisements.

6.3 Role Permissions

If there is a conflict between the permissions described here and those defined earlier, the permissions specified here take precedence.

VISITOR (**Visitors**)

Visitors are unauthenticated users. They may perform any actions allowed to everyone.

FREEUSER (**Free users**)

Free users can perform any actions allowed for authenticated users.

PREMIUMUSER (**Premium users**)

Premium users have all the permissions of free users. In addition, they are allowed to create free play sessions.

TRAINER (**Trainers**)

Trainers have all the permissions of premium users. Additionally, they can create training sessions and may be assigned as trainers of sessions.

ADMIN (Administrators)

Administrators have the same permissions as premium users. In addition, they can read any user's core information and email address; and edit any user's role. They also manage sessions: they can edit any session's core information and delete any session.

7 Privacy Requirements

We focus on data protection as a specific kind of privacy requirement. In general, these requirements ensure that a user's personal data is processed according to their data-usage policy. Since users rarely specify such policies explicitly, data protection regulations, such as the General Data Protection Regulation (GDPR), mandate that certain baseline data-usage policies be enforced.

In this project, we concentrate on the two key data-usage policies (from the GDPR):

- Purpose-limitation (Art. 5 §1 (b) GDPR): Personal data shall be collected for specified, explicit, and legitimate purposes and not further processed in a manner that is incompatible with those purposes.
- Data-subject consent (Art. 7 §1 GDPR): Where processing is based on consent, the controller shall be able to demonstrate that the data subject (i.e., the user) has consented to the processing of his or her personal data.

7.1 Personal data

In the SportLink application, the following are considered personal data:

- In the **Person** entity: the `name`, `surname`, `role`, and `email` attributes, as well as the subscriptions and the registers association ends.
- In the **Registration** entity: the `registered_at` and the `session` association end.

Note: For an instance of the **Person** entity, the data owner is the person themselves. For an instance of the **Registration** entity, the data owner is the user who created the registration.

7.2 Purposes

In this application, we define the following set of purposes for processing personal data:

- Any: This is the most general, top-level purpose. This purpose contains `Marketing`, `Analytics`, and `Functional` purposes.
- Marketing: This includes specific types of marketing and is divided into:
 - `TargetedMarketing`: Related to displaying personalized advertisements to users on their profile page.
 - `MassMarketing`: Allows administrators to send generic advertisements to all users, regardless of subscription or activity.
- Analytics: This focuses on analyzing users' data to generate insights and statistics. Specifically, an administrator may analyze session attendance by sport or compare free play versus training popularity.
- Functional: This represents specific functional purposes related to core platform features that support session management and sport participation. It includes:
 - `Recommendation`: personalized session suggestions on the user's main page of the application.
 - `Management`: user-related management functionality.

- * `SessionManagement`: sessions created or joined by the user.
- * `UserManagement`: assigning or updating trainers for training sessions.
- **Core**: Any other functionality of the platform not mentioned above is considered core functionality.

Whenever any personal data is used (actions: read, add, remove, or update) **as part of the execution of an application's function that implements the functionality associated with a purpose above**, that purpose will be considered the **actual purpose** for the data use.

For example, suppose function `func` (implementing the functionality for the P1 purpose) calls a function `foo` that accesses a name of a **Person**. The actual purpose of the data access is P1. If `foo` is called by a different function, the actual purpose may be different. If a function `bar` implementing functionality associated with purpose P1 calls a function `baz` implementing functionality associated with purpose P2 any personal data access within `baz`'s execution will have both P1 and P2 as actual purposes.

In your implementation, you have to **keep track of the actual purpose** in order to use it to check compliance with the privacy policy.

7.3 Privacy policy

A privacy policy (or notice) is a list of declared purposes (a personal data list, a purpose, and a constraint triple) shown to the user. The application's privacy policy is as follows:

- your name for Marketing Purpose.
- your name, surname, role, subscriptions, and email for Core Purpose.
- your subscriptions and registers for Recommendation Purpose, *if you are a Free user*.
- your registers for TargetedMarketing Purpose.
- your email for MassMarketing Purpose.
- your registers for Analytics Purpose.

Users may grant or revoke consent for each declared purpose. If the constraint is not specified in the declared purpose, the if true constraint is assumed. The state of a user's consent is stored in the database using the **Consent** entity, which includes the class of personal data `classname`, the specific personal data field `propertyname`, the purpose for which the data usage is declared, and the data owner `user`.

To implement the privacy requirements, one must:

1. Present the above privacy policy to the user and allow them to grant or revoke consent (see the declared purposes part of the *νActionGUI*'s privacy model)
2. Check if a personal data use is allowed for a set of actual purposes by checking if:
 - all the actual purposes are declared in the privacy policy for that data, and
 - the data owner consented to all of the actual purposes (or to purposes containing the actual purposes) for that data.